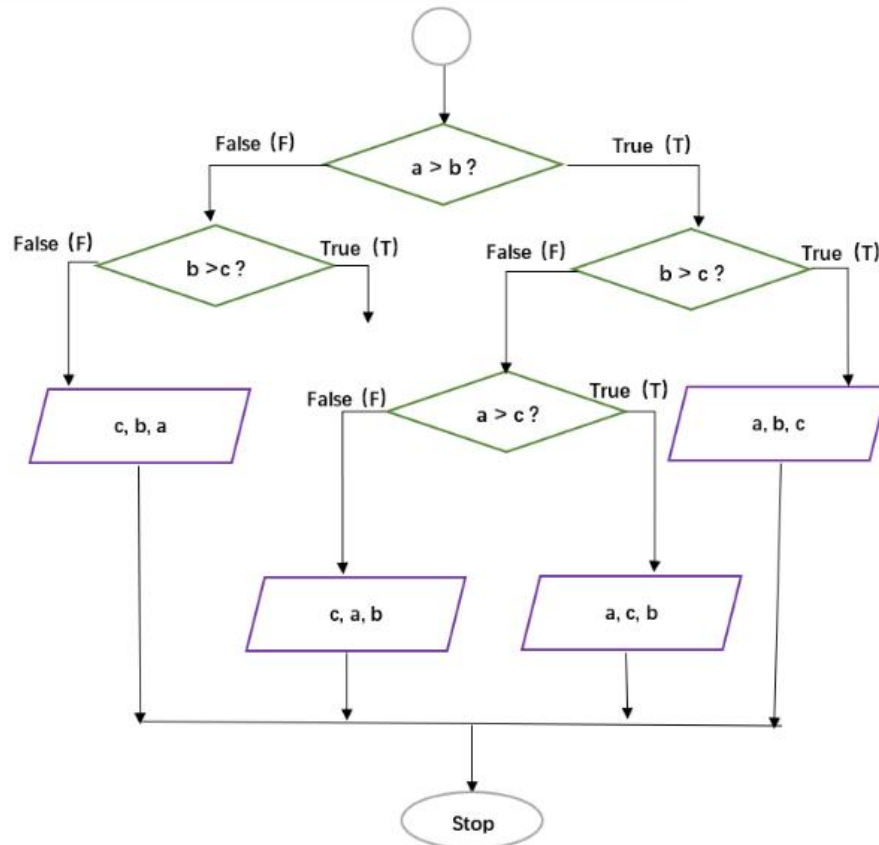


1. Flowchart

[10 points] Write a function `Print_values` with arguments `a`, `b`, and `c` to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random `a`, `b`, and `c` values.



● 参考资料

阅读《python》- “更多控制流工具->if 语句” 给了我启发

4.1. if 语句

最让人耳熟能详的语句应当是 `if` 语句：

```
>>> x = int(input("Please enter an integer: "))
Please enter an integer: 42
>>> if x < 0:
...     x = 0
...     print('Negative changed to zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Single')
... else:
...     print('More')
...
More
```

可有零个或多个 `elif` 部分，`else` 部分也是可选的。关键字 `'elif'` 是 `'else if'` 的缩写，用于避免过多的缩进。 `if ... elif ... elif ...` 序列可以当作其它语言中 `switch` 或 `case` 语句的替代品。

如果是把一个值与多个常量进行比较，或者检查特定类型或属性，`match` 语句更有用。详见 [match 语句](#)。

阅读《菜鸟教程》- “python3 高级教程->Python 随机数生成” 给了我启发

Python 随机数生成

Python3 实例

在 Python 中，可以使用内置的 random 模块来生成随机数。

```
import random
```

random.random()

random.random() 返回一个介于 0.0 和 1.0 之间的随机小数：

实例

```
import random

random_number = random.random()

print(random_number)
```

执行以上代码输出结果为：

```
0.7597072251250637
```

random.randint(a, b)

random.randint(a, b) 用于返回一个介于 a 和 b 之间的整数（包括 a 和 b）。

```
random.randint(a,b)
```

函数返回数字 N，N 为 a 到 b 之间的数字（a <= N <= b），包含 a 和 b。

● 代码

```
#1.流程图
import random
a=random.randint(0,100)
b=random.randint(0,100)
c=random.randint(0,100)

def Deter_size(a, b, c):
    if a > b:
        if b > c:
            return a, b, c
        else:
            if a > c:
                return a, c, b
            else:
                return c, a, b
    else:
        if a > c:
            return b, a, c
        else:
            if b > c:
                return b, c, a
            else:
                return c, b, a

result = Deter_size(a, b, c)
print(f"a={a}, b={b}, c={c}")
print(f"{result}")
```

➤ Deter_size(a, b, c)通过嵌套 if 比较三个数，再打印对应次序。

- 结果

Name	Type	Size	
a	int	1	94
b	int	1	92
c	int	1	50
result	tuple	3	(94, 92, 50)

```
In [6]: print(f"a={a}, b={b}, c={c}")
a=94, b=92, c=50

In [7]: print(f"{result}")
(94, 92, 50)
```

- 通过输出结果发现是以降序排列。

2. Matrix multiplication

2.1 [5 points] Make two matrices M1 (5 rows and 10 columns) and M2 (10 rows and 5 columns); both are filled with random integers from 0 and 50.

2.2 [10 points] Write a function Matrix_multip to do matrix multiplication, i.e., $M1 * M2$. Here you are ONLY allowed to use for loop, * operator, and + operator.

- 参考资料

阅读《菜鸟教程》 - “python3 高级教程->Python 随机数生成” 给了我启发

Python 随机数生成

Python3 实例

在 Python 中, 可以使用内置的 random 模块来生成随机数。

```
import random
```

random.random()

random.random() 返回一个介于 0.0 和 1.0 之间的随机小数:

实例

```
import random

random_number = random.random()
print(random_number)
```

执行以上代码输出结果为:

```
0.7597072251250637
```

random.randint(a, b)

random.randint(a, b) 用于返回一个介于 a 和 b 之间的整数 (包括 a 和 b) 。

```
random.randint(a,b)
```

函数返回数字 N, N 为 a 到 b 之间的数字 (a <= N <= b), 包含 a 和 b。

- 代码

```
#2.1
import numpy as np

M1=np.random.randint(0,50,size=(5,10))
M2=np.random.randint(0,50,size=(10,5))
```

- 用 random.randint(0, 50)生成 M1 和 M2。M1 为 5×10 , M2 为 10×5 , 因此乘积为 5×5 。

```
#2.2
def Matrix_multip(a, b):
    rows = a.shape[0]
    cols = b.shape[1]
    k_len = a.shape[1]

    arr = np.zeros((rows, cols))

    # 遍历结果的每一行
    for i in range(rows):
        # 遍历结果的每一列
        for j in range(cols):
            # 把第 i 行和第 j 列对应位置相乘再累加
            s = 0
            for k in range(k_len):
                s += a[i][k] * b[k][j]
            arr[i][j] = s

    return arr

result = Matrix_multip(M1, M2)
print("结果形状: ", result.shape)
print(result)
```

➤ Matrix_multip 使用三层 for 循环；每个结果元素先设为 0，再通过 “*” 和 “+” 累加。

● 结果

Name	Type	Size		In [11]: print(result)
M1	Array of int32	[5, 10]	[[24 30 22 ... 44 45 34] [27 5 31 ... 4 7 5]	[[9276. 7591. 6750. 9955. 6367.] [4569. 5315. 5148. 5636. 4936.]
M2	Array of int32	[10, 5]	[[37 37 9 33 29] [29 9 2 35 6]	[6507. 7329. 6258. 7406. 6017.] [9951. 8367. 6925. 11016. 7403.]
result	Array of float64	[5, 5]	[[9276. 7591. 6750. 9955. 6367.] [4569. 5315. 5148. 5636. 4 ...]	[5992. 5593. 4843. 6451. 3979.]

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function Pascal_triangle with an argument k to print the kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200).

二项式系数表：

k\ n	1	2	3	4	5	6	7	8	9
1	1								
2	1	1							
3	1	2	1						
4	1	3	3	1					
5	1	4	6	4	1				
6	1	5	10	10	5	1			
7	1	6	15	20	15	6	1		
8	1	7	21	35	35	21	7	1	
9	1	8	28	56	70	56	28	8	1

● 参考资料

阅读《MathWorld》->“Pascal's Triangle”给了我启发，用于杨辉三角与组合数的关系。

<https://mathworld.wolfram.com/PascalsTriangle.html>

● 代码

```
#(1)Numpy
import numpy as np

def Pascal_triangle(k):
    arr = np.zeros((k, k))

    arr[:, 0] = 1

    for i in range(1, k):
        arr[i][i] = 1

        for j in range(1, i):
            arr[i][j] = arr[i-1][j-1] + arr[i-1][j]

    return arr

k_100 = Pascal_triangle(100)
for i in range(100):
    print(k_100[i, :i+1])

k_200=Pascal_triangle(200)
```

```
#(2)列表嵌套
def Pascal_triangle_1(k):
    triangle = []

    for i in range(k):
        row = [1] * (i + 1) #行

        for j in range(1, i):
            row[j] = triangle[i-1][j-1] + triangle[i-1][j] #列

        triangle.append(row)

    return triangle

k1_100 = Pascal_triangle_1(100)
for row in k1_100:
    print(row)
```

- 三角首行为[1]，其后每一行首尾元素均为 1，中间位置的每个元素由上一行相邻两项相加得到。
- NumPy 版本通过创建 $k \times k$ 的零矩阵存储结果，先将第一列与对角线元素初始化为 1，再通过双层循环逐行逐列计算中间元素的值。列表嵌套版本则逐行构造列表，每一行先以全 1 初始化，随后利用上一行的数据更新中间位置，最终将每行追加至整体列表中。

● 结果



4. Add or double

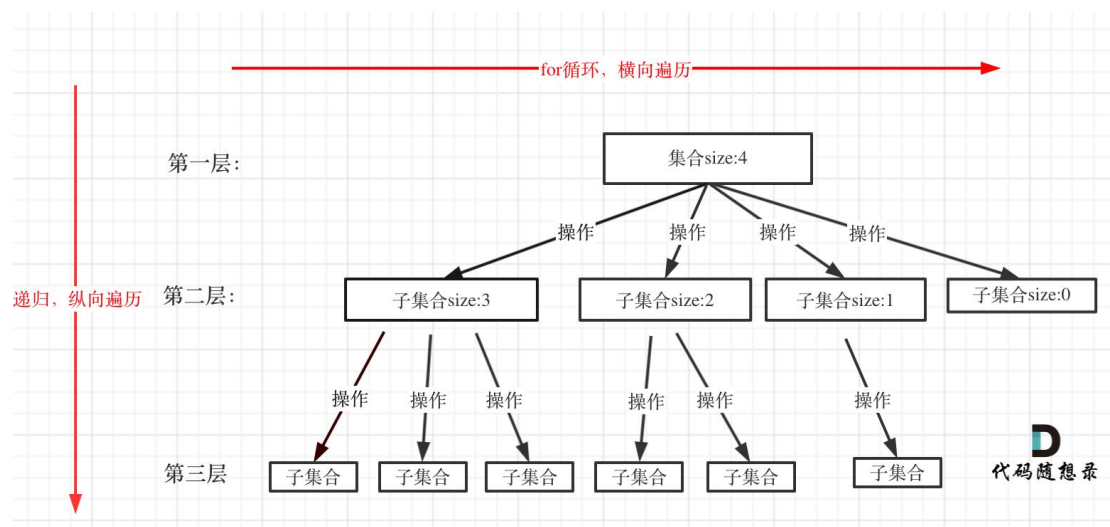
[20 points] If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly x RMB? Here x is an integer randomly selected from 1 to 100. Write a function `Least_moves` to print your results. For example, `Least_moves(2)` should print 1, and `Least_moves(5)` should print 3.

● 参考资料

阅读《代码随想录》- “回溯算法” 给了我启发

贪心算法一般分为如下四步：

- ❖ 将问题分解为若干个子问题
- ❖ 找出适合的贪心策略
- ❖ 求解每一个子问题的最优解
- ❖ 将局部最优解堆叠成全局最优解



● 代码

```
#4. 加1或者翻倍
def Least_moves(x):
    move = 0
    current = x
    while current > 1:
        if current % 2 == 1: #奇数, 逆向只能减1
            current = current - 1
        else: #偶数, 逆向除以2
            current = current // 2
        move += 1
    return move

print(Least_moves(2))
print(Least_moves(5))
```

- 函数 `Least_moves(x)` 求解从初始 1 元, 通过每次加 1 或者翻倍操作, 凑出 x 元的最小操作步数。

- 算法采用逆向推导思想：从目标值 x 反向还原到 1。若当前数为奇数，则上一步只能是加 1 操作，因此当前值减 1；若当前数为偶数，则上一步最优选择是翻倍操作，因此当前值除以 2。每进行一次还原操作，步数计数加 1，直到数值变为 1，最终返回总步数。

● 结果

```
In [29]: print(Least_moves(2))
1

In [30]: print(Least_moves(5))
3
```

5. Dynamic programming

Insert + or - operation anywhere between the digits 123456789 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function `Find_expression`, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, `Find_expression(50)` should print lines include:

$1-2+34+5+6+7+8-9=50$

and

$1+2+34-56+78-9=50$

5.2 [5 points] Count the total number of suitable solutions for any integer i from 1 to 100, assign the count to a list called `Total_solutions`. Plot the list `Total_solutions`, so which number(s) yields the maximum and minimum of `Total_solutions`?

● 参考资料

阅读《matplotlib》- “matplotlib.pyplot.plot” 给了我启发，用于解数折线图绘制。

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

matplotlib.pyplot.plot

`matplotlib.pyplot.plot(*args, scalex=True, scaley=True, data=None, **kwargs)`

Plot y versus x as lines and/or markers.

[\[source\]](#)

Call signatures:

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

The coordinates of the points or line nodes are given by x, y .

The optional parameter `fmt` is a convenient way for defining basic formatting like color, marker and linestyle. It's a shortcut string notation described in the *Notes* section below.

```
>>> plot(x, y)           # plot x and y using default line style and color
>>> plot(x, y, 'bo')      # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')         # ditto, but with red plusses
```

You can use [Line2D](#) properties as keyword arguments for more control on the appearance. Line properties and `fmt` can be mixed. The following two calls yield identical results:

- 代码

```
#5.1
def Find_expression(target):
    digits = "123456789" # 固定的数字串, 顺序不能变
    results = [] # 存所有符合条件的表达式

    # pos: 当前处理到第几个数字 (从0开始)
    # expr: 当前表达式字符串
    # total: 当前表达式算出来的值
    def dfs(pos, expr, total):
        # 边界条件: 9个数字都处理完了
        if pos == len(digits):
            # 如果结果等于目标值, 就存起来
            if total == target:
                results.append(expr)
            return

        # 从pos开始, 尝试取1个、2个、...、直到最后一个数字, 拼成一个数
        # 比如pos=1时, end可以是2、3、...、9
        # 对应拼成: 2、23、234、...、23456789
        for end in range(pos + 1, len(digits) + 1):
            num_str = digits[pos:end] # 取出数字字符串
            num = int(num_str) # 转成整数

            if pos == 0:
                # 第一个数字, 前面没有运算符, 直接拼
                dfs(end, num_str, num)
            else:
                # 选择1: 前面加 +
                dfs(end, expr + "+" + num_str, total + num)
                # 选择2: 前面加 -
                dfs(end, expr + "-" + num_str, total - num)

        # 从第0个数字开始搜
        dfs(0, "", 0)

    # 打印所有结果
    for line in results:
        print(f"{line}={target}")
```

- 函数 Find_expression(target) 基于深度优先搜索实现, 从数字串 123456789 的首位开始逐位递归。每到一个间隙, 依次尝试插入加号、插入减号或与后续数字拼接成多位数, 实时更新表达式字符串与计算结果。遍历完所有数字后, 若结果等于目标值则将该表达式存入结果列表并打印。

```
#5.2
from collections import defaultdict
import matplotlib.pyplot as plt

# 解决中文显示问题
plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

def count_all_solutions():
    digits = "123456789"
    count = defaultdict(int) # key是数字, value是有多少种表达式能算出它

    def dfs(pos, expr, total):
        if pos == len(digits):
            count[total] += 1 # 每走完一种完整的表达式, 计数+1
            return

        for end in range(pos + 1, len(digits) + 1):
            num_str = digits[pos:end]
            num = int(num_str)

            if pos == 0:
                dfs(end, num_str, num)
            else:
                dfs(end, expr + "+" + num_str, total + num)
                dfs(end, expr + "-" + num_str, total - num)

        dfs(0, "", 0)

    return count
```

```

# ===== 主程序 =====
print("正在统计所有方案...")
count = count_all_solutions()

# 提取1~100的方案数, 存入 Total_solutions 列表
Total_solutions = []
for target in range(1, 101):
    Total_solutions.append(count.get(target, 0))

print(f"统计完成! 一共 {len(count)} 个不同的数字可以被表示出来。")

# ===== 找最大值和最小值 =====
max_count = max(Total_solutions)
min_count = min(Total_solutions)

max_targets = [i + 1 for i, c in enumerate(Total_solutions) if c == max_count]
min_targets = [i + 1 for i, c in enumerate(Total_solutions) if c == min_count]

print(f"\n最多方案数: {max_count} 种")
print(f"对应数字: {max_targets}")
print(f"\n最少方案数: {min_count} 种")
print(f"对应数字: {min_targets}")

# ===== 画图 =====
plt.figure(figsize=(12, 6))
plt.plot(range(1, 101), Total_solutions, 'b-', linewidth=1)

# 标出最大值
first_max = True
for target in max_targets:
    if first_max:
        plt.scatter(target, Total_solutions[target - 1],
                    c='red', s=50, zorder=5, label='Maximum')
        first_max = False
    else:
        plt.scatter(target, Total_solutions[target - 1],
                    c='red', s=50, zorder=5)

# 标出最小值
first_min = True
for target in min_targets:
    if first_min:
        plt.scatter(target, Total_solutions[target - 1],
                    c='green', s=50, zorder=5, label='Minimum')
        first_min = False
    else:
        plt.scatter(target, Total_solutions[target - 1],
                    c='green', s=50, zorder=5)

plt.xlabel('Target integer')
plt.ylabel('Number of solutions')
plt.title('Expression solutions for targets 1 to 100')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

```

- 在搜索过程中同步统计每个计算结果对应的表达式数量, 将 1 至 100 各数字的方案数存入列表 Total_solutions。随后调用 matplotlib 绘制折线图展示分布, 并遍历列表找出方案数最多与最少的对应数字, 完成对所有合法表达式的计数与可视化分析。

● 结果

